



INDIA.RUNS

Build what next India runs on

Team Name : phenoX

Team Leader Name : Jatin Jain

Problem Statement : Intelligent Candidate Discovery & Ranking (Top 100 / 100k)

Solution Overview

- What is your proposed solution?
 - A fast, two-step offline ranking system. First, a pre-built Parquet file quickly filters candidates. Then, JSONL streaming does deeper scoring on the shortlisted set.
- What differentiates your approach?
 - Speed: Ranks 100k candidates in under 10 seconds on a normal CPU, no GPU needed.
 - No Made-Up Answers: Uses fixed, rule-based logic plus pre-generated AI reasoning, so results stay consistent and explainable.
 - Looks Beyond Resumes: Considers behavior like response rate and notice period, not just keyword matching.

JD Understanding & Candidate Evaluation

❖ What are the key requirements extracted?

- Experience: 5–9 years in applied ML/NLP (the ideal range).
- Tech Skills: Vector databases (Pinecone, Qdrant) and LLM tools (LoRA, Sentence Transformers).

❖ How does your solution evaluate fit?

- Skill Quality Score: Combines how relevant the skill is, how good the candidate is at it, how long they've used it, and endorsements from others.
- Behavior Score: Lowers the score for slow responders or long notice periods, and boosts it for active GitHub contributors.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**
 - Retrieve: Uses a pre-built Parquet file to instantly filter 100k candidates down to the top 2,000.
Score: Streams full profile data for those 2,000 and scores each on 6 factors. Rank: Sorts all scored candidates and selects the top 100.
- **What models, algorithms, or heuristics are used?**
 - No ML model, a transparent rule-based heuristic so results stay explainable.
 - Base Score (out of 100): Title (30) + Skills (25) + Experience (15) + Company (15) + Location (10).
Behavior Multiplier: Applied on top, ranging from 0.40x to 1.15x.
- **How are multiple candidate signals combined into a final ranking?**
 - Job-fit signals (title, skills, experience, company, location) add up to a base score out of 100. Behavioral signals (response rate, notice period, GitHub activity) combine into a single multiplier. Final Score = Base Score x Behavior Multiplier.

Explainability & Data Validation

❖ How are ranking decisions explained?

Gemini 2.5 Flash generates the reasoning offline, before ranking runs, not live.

A template then fills in each candidate's real stats (years of experience, matched skills) for a personalized explanation.

❖ How do you prevent hallucinations or unsupported justifications?

No live AI calls, all explanations are based strictly on the candidate's real profile data.

Since reasoning is pre-generated and cached, every output can be checked against the source data before use.

❖ How does your solution handle inconsistent, low-quality, or suspicious profiles?

A 5-point honeypot check runs before ranking and flags suspicious profiles, like 20 years of experience at age 22.

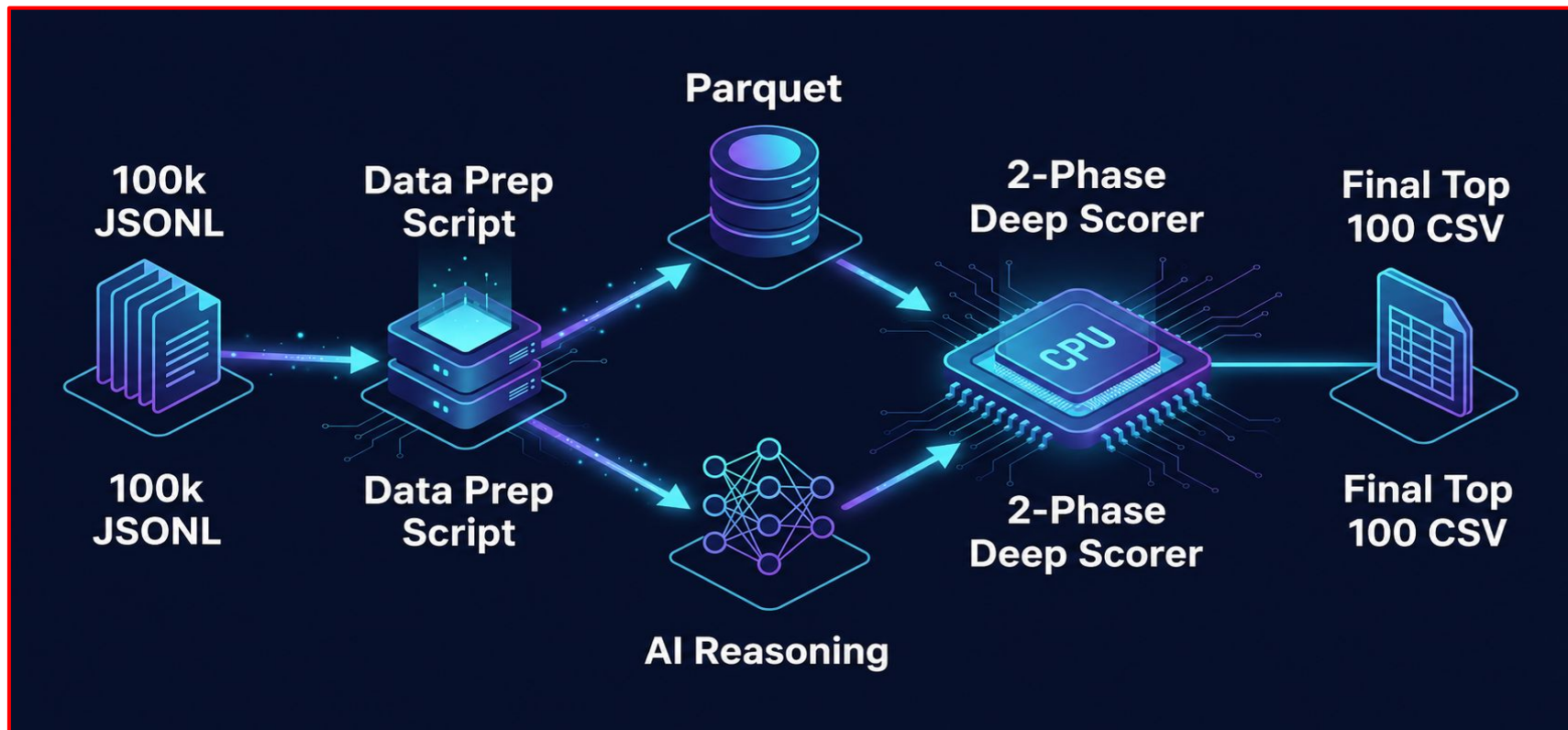
Flagged and low-quality profiles are removed early, so they never reach the top 100.

End-to-End Workflow

- What is the complete workflow?

- ❖ 1. Offline Setup: Read all 100k raw profiles once, run the honeypot check to flag fake/suspicious ones, then save the clean data as a Parquet file. This is a one-time step, so it doesn't slow down the actual ranking later.
- ❖ 2. Fast Filter: When a new job description comes in, load the Parquet file and instantly drop the 98% of candidates who clearly don't match (wrong experience range, missing core skills, etc.). This narrows 100k down to about 2,000 in seconds.
- ❖ 3. Deep Scoring: Stream through the full profile data of those 2,000 candidates one at a time, calculating the 6-factor score and behavior multiplier for each. Streaming (instead of loading everything at once) keeps memory usage low even on a large dataset.
- ❖ 4. Final Output: Sort the 2,000 scored candidates, take the top 100, attach their pre-cached reasoning, and export everything as a CSV, all in about 10 seconds total.

System Architecture



Results & Performance

- What results demonstrate ranking quality?

The top 10 candidates all match the 5–9 years experience range, hold titles like "Senior AI Engineer," and have hands-on Vector DB experience. No fake profiles made it into the top 100.

- How does it meet compute constraints?

Speed: Runs in about 10 seconds total (well under the 5-minute limit).

Compute: Runs fully offline, no internet calls, no GPU needed.

Memory: Stays efficient throughout, peeking under 4 GB of RAM.

Technologies Used

- What technologies were used and why?

Python (Generators): Reads large JSONL files one record at a time, so 100k profiles can be processed without loading everything into memory.

Pandas & Parquet: Stores data by column instead of by row, so filtering by experience or skills is much faster than checking each profile one by one.

Gemini 2.5 Flash API: Used offline, before ranking runs, to write natural, easy-to-read reasoning for each candidate. No live calls means no added delay and no risk of made-up answers.

Gradio / HuggingFace: Wraps the ranking system into a simple web demo, so anyone can test it with their own job description without touching code.

Submission Assets

GitHub Repository : [IndiaRunsSubmission-Candidate-Ranker](#)

Gradio Sandbox Demo: [Redrob Candidate Ranker UI](#)

Final Output File : team_submission.csv (attached to submission)



INDIA.RUNS

Build what next India runs on

THANK YOU